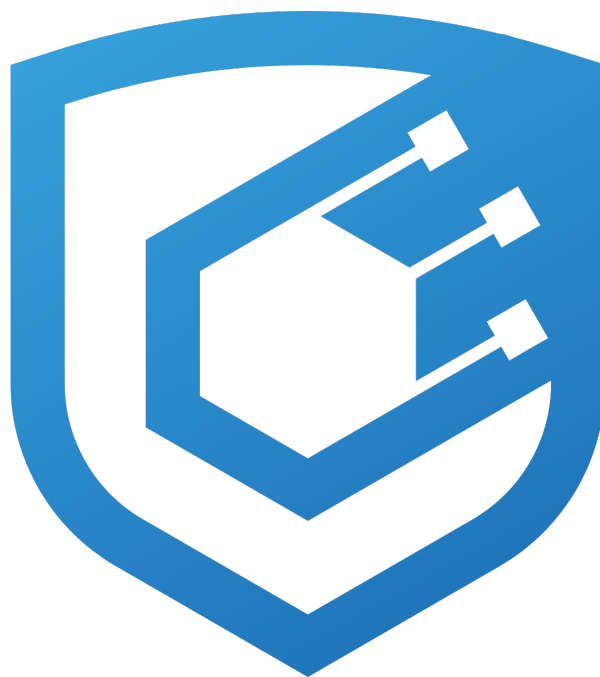




Securitize Tempo TIP-20 Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Kage](#)

[Immeas](#)

August 28, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Centralization Risks	3
6.1.1	Inherent Admin Powers	3
7	Findings	6
7.1	Medium Risk	6
7.1.1	Tip20RegistryService::removeWallet lets registrars bypass active investor locks	6
7.2	Low Risk	8
7.2.1	Tip20RegistryService::policyId can drift from the token transfer policy	8
7.2.2	deploy-full.ts uses the wrong TIP-20 renounceRole signature	9
7.2.3	deploy-full.ts predicts the TIP-403 policy ID instead of reading the receipt	10
7.2.4	Tip20TrustService::setServiceOwner can assign MASTER to an inert address	10
7.2.5	Tip20RegistryService::setPolicyId accepts built-in TIP-403 policies	11
7.2.6	Tip20TrustService::setServiceOwner uses one-step ownership transfer	12
7.2.7	Tip20RegistryService::registerInvestorWithWallets exceeds Tempo's transaction gas cap	13
7.2.8	loadPk reads deployer keys from plaintext sources	13
7.3	Informational	15
7.3.1	Tip20TrustService::_project can revoke unmanaged native grants	15
7.3.2	Tip20RegistryService::isAuthorized documentation does not match policy state	16
7.3.3	Tip20RegistryService events do not expose effective wallet authorization	16
7.3.4	Tip20RegistryService::setServiceConsumer missing event	17
7.3.5	Tip20TrustService::_setRole emits abstract-role events after native calls	17
7.3.6	Tip20RegistryService::registerInvestorWithWallets can retain stale proof metadata	17
7.3.7	deploy-full.ts permits overlapping operational role addresses	17
7.3.8	deploy-full.ts blindly retries one-shot deployment operations	18
7.3.9	deploy-full.ts suppresses investor onboarding failures	18
7.3.10	deploy-full.ts does not verify the target Tempo network	18
7.3.11	ITip20TrustService misdocuments MASTER native permissions	19
7.3.12	Tip20RegistryService::addPlatformWallet has an unreachable duplicate check	19
7.3.13	Tip20RegistryService storage-gap comment has the wrong slot count	19
7.3.14	Tip20RegistryService::registerInvestorWithWallets, isAuthorized mishandle Tempo virtual addresses	19
7.3.15	deploy-full.ts can never execute mint step	20
7.3.16	Tip20RegistryService::initialize does not reject a zero admin	20

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Securitize's TIP-20 service layer adds regulated-securities compliance to native Tempo tokens. Tempo supplies transfer permissioning through TIP-403 whitelist policies and native roles for minting, burning, pausing and seizing, but does not model investors, multi-wallet identities, per-investor freezes or Securitize's abstract role scheme. Three upgradeable contracts bridge that gap: `Tip20RegistryService` records investor identities, wallets and compliance attributes, then projects wallet eligibility into TIP-403; `Tip20TrustService` maps one abstract role per account into token and registry roles; and `Tip20ServiceConsumer` binds the two services by ID.

All three services are UUPS implementations behind ERC-1967 proxies and hold no user balances. The asset is a native TIP-20 created through Tempo's factory predeploy, while the service layer supplies the investor and role controls around it. A deployment can be discovered from its token address through the transfer policy, registry, service consumer and trust service.

5 Audit Scope

The audit scope was limited to:

```
// contracts
contracts/registry/Tip20RegistryService.sol
contracts/service/Tip20ServiceConsumer.sol
contracts/trust/Tip20TrustService.sol

// deployment
ignition/modules/Tip20RegistryService.ts
scripts/deploy-full.ts
scripts/storage-layout.js
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [Tempo TIP-20](#) code provided by [Securitize](#). The findings consist of 1 Medium & 8 Low severity issues with the remainder being informational.

6.1 Centralization Risks

6.1.1 Inherent Admin Powers

- Upgrade control spans independent authorization planes with no on-chain timelock, multisig requirement or restriction on replacement code. `Tip20RegistryService::_authorizeUpgrade` is controlled by its `DEFAULT_ADMIN_ROLE`, while `Tip20TrustService::_authorizeUpgrade` and `Tip20ServiceConsumer::_authorizeUpgrade` depend on the trust's `MASTER` role. An authorized upgrade can rewrite the investor freeze, wallet-registration rules and role projection.
- Transferring `MASTER` does not transfer or revoke `DEFAULT_ADMIN_ROLE` on the token, registry or service consumer. Native root holders can use the break-glass role-administration path, and their authority is not represented by `Tip20TrustService::getRole`.
- `DEFAULT_ADMIN_ROLE` can change which role governs projected registry permissions through `Tip20RegistryService::setRoleAdmin`, selecting who may grant and revoke those permissions.
- `MASTER` has the union of native mint, pause, unpause and blocked-burn capabilities plus both registry roles. `ISSUER` can mint and perform blocked burns, and also receives both registry roles. These powers can affect investor balances and transfer eligibility.
- `LOCK_MANAGER_ROLE` can freeze or unfreeze any registered investor, de-authorizing every wallet it holds on TIP-403 and blocking its transfers until unfrozen.
- `REGISTRAR_ROLE` controls investor onboarding and wallet authorization. It can also add or remove identity-less platform wallets, which are outside investor locks and can therefore be authorized or de-authorized independently of the investor model.

Summary

Project Name	Tempo TIP-20
Repository	bc-tempo-sc
Commit	e70159bfcf3c...
Fix Commit	824107547d1e...
Audit Timeline	Aug 12th - Aug 18th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	8
Informational	16
Gas Optimizations	0
Total Issues	25

Summary of Findings

[M-1] Tip20RegistryService::removeWallet lets registrars bypass active investor locks	Resolved
[L-1] Tip20RegistryService::policyId can drift from the token transfer policy	Resolved
[L-2] deploy-full.ts uses the wrong TIP-20 renounceRole signature	Resolved
[L-3] deploy-full.ts predicts the TIP-403 policy ID instead of reading the receipt	Resolved
[L-4] Tip20TrustService::setServiceOwner can assign MASTER to an inert address	Resolved
[L-5] Tip20RegistryService::setPolicyId accepts built-in TIP-403 policies	Resolved
[L-6] Tip20TrustService::setServiceOwner uses one-step ownership transfer	Resolved
[L-7] Tip20RegistryService::registerInvestorWithWallets exceeds Tempo's transaction gas cap	Resolved
[L-8] loadPk reads deployer keys from plaintext sources	Partially Resolved
[I-01] Tip20TrustService::_project can revoke unmanaged native grants	Acknowledged
[I-02] Tip20RegistryService::isAuthorized documentation does not match policy state	Resolved
[I-03] Tip20RegistryService events do not expose effective wallet authorization	Resolved
[I-04] Tip20RegistryService::setServiceConsumer missing event	Resolved
[I-05] Tip20TrustService::_setRole emits abstract-role events after native calls	Resolved
[I-06] Tip20RegistryService::registerInvestorWithWallets can retain stale proof metadata	Resolved
[I-07] deploy-full.ts permits overlapping operational role addresses	Resolved
[I-08] deploy-full.ts blindly retries one-shot deployment operations	Acknowledged
[I-09] deploy-full.ts suppresses investor onboarding failures	Resolved

[I-10] <code>deploy-full.ts</code> does not verify the target Tempo network	Resolved
[I-11] <code>ITip20TrustService</code> misdocuments MASTER native permissions	Resolved
[I-12] <code>Tip20RegistryService::addPlatformWallet</code> has an unreachable duplicate check	Resolved
[I-13] <code>Tip20RegistryService</code> storage-gap comment has the wrong slot count	Resolved
[I-14] <code>Tip20RegistryService::registerInvestorWithWallets</code> , <code>isAuthorized</code> mishandle Tempo virtual addresses	Resolved
[I-15] <code>deploy-full.ts</code> can never execute mint step	Resolved
[I-16] <code>Tip20RegistryService::initialize</code> does not reject a zero admin	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Tip20RegistryService::removeWallet lets registrars bypass active investor locks

Description: Tip20RegistryService::lockInvestor deauthorizes an investor's wallets, but the freeze is stored only on the investor. A registrar can call removeWallet, then attach the same address to an unlocked investor or call addPlatformWallet. Either path reauthorizes the address in TIP-403 while the original investor remains locked.

Impact: An account with REGISTRAR_ROLE, including an EXCHANGE, can release a frozen wallet without LOCK_MANAGER_ROLE. The wallet can transfer its blocked balance while registry monitoring still reports the original investor as locked.

Proof of Concept: Add the following test to test/solace-pocs/ActiveInvestorLockBypass.test.ts:

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";

const POLICY_ID = 2n;

async function fixture() {
  const [admin, registrar, lockManager, frozenWallet] = await ethers.getSigners();
  const tip403 = await ethers.deployContract("MockTIP403");
  const Registry = await ethers.getContractFactory("Tip20RegistryService");
  const implementation = await Registry.deploy();
  const proxy = await ethers.deployContract("ERC1967Proxy", [
    await implementation.getAddress(),
    Registry.interface.encodeFunctionData("initialize", [admin.address, await tip403.getAddress()]),
  ]);
  const registry: any = Registry.attach(await proxy.getAddress());

  await registry.setPolicyId(POLICY_ID);
  await registry.grantRole(await registry.REGISTRAR_ROLE(), registrar.address);
  await registry.grantRole(await registry.LOCK_MANAGER_ROLE(), lockManager.address);
  await registry.connect(registrar).registerInvestor("INV-1");
  await registry.connect(registrar).addWallet(frozenWallet.address, "INV-1");
  await registry.connect(lockManager).lockInvestor("INV-1");

  return { registry, tip403, registrar, frozenWallet };
}

describe("Active investor lock bypass", () => {
  it("re-authorizes a frozen wallet by attaching it to an unlocked investor", async () => {
    const { registry, tip403, registrar, frozenWallet } = await loadFixture(fixture);

    expect(await registry.isInvestorLocked("INV-1")).to.equal(true);
    expect(await tip403.isAuthorized(POLICY_ID, frozenWallet.address)).to.equal(false);

    await registry.connect(registrar).removeWallet(frozenWallet.address);
    await registry.connect(registrar).registerInvestor("INV-2");
    await registry.connect(registrar).addWallet(frozenWallet.address, "INV-2");

    expect(await tip403.isAuthorized(POLICY_ID, frozenWallet.address)).to.equal(true);
    expect(await registry.isInvestorLocked("INV-1")).to.equal(true);
    expect(await registry.getInvestor(frozenWallet.address)).to.equal("INV-2");
  });

  it("re-authorizes a frozen wallet outside the identity model", async () => {
    const { registry, tip403, registrar, frozenWallet } = await loadFixture(fixture);
```

```
    await registry.connect(registrar).removeWallet(frozenWallet.address);
    await registry.connect(registrar).addPlatformWallet(frozenWallet.address);

    expect(await tip403.isAuthorized(POLICY_ID, frozenWallet.address)).to.equal(true);
    expect(await registry.isWallet(frozenWallet.address)).to.equal(false);
    expect(await registry.isPlatformWallet(frozenWallet.address)).to.equal(true);
    expect(await registry.isInvestorLocked("INV-1")).to.equal(true);
  });
});
```

Run with: `npx hardhat test test/solace-pocs/ActiveInvestorLockBypass.test.ts`

Recommended Mitigation: Consider preventing removal only while the frozen wallet still holds tokens. Once its balance is zero, it may be removed and reused without lock-manager approval.

Securitize: Fixed in [PR 14](#).

Cyfrin: Partially resolved. PR 14 addresses the main issue. . A residual ordering-dependent scenario remains where a compromised registrar could remove the funded wallet before the lock and rehome it afterward, allowing it to remain usable. The team has acknowledged and accepted this risk.

Securitize: Fixed in [PR 20](#)

Cyfrin: Resolved. PR 20 closes the ordering path by reading the balance on every removal rather than only under an active lock, and requiring LOCK_MANAGER_ROLE to detach a funded wallet from an unlocked investor.

7.2 Low Risk

7.2.1 Tip20RegistryService::policyId can drift from the token transfer policy

Description: Tip20RegistryService::policyId is immutable after setup, but the token administrator can change the token's effective policy. Registry writes and isAuthorized then use the old policy. Since Tempo T9, the effective token binding can be queried through TIP-403 tokenTransferPolicyId ([TIP-1092](#)).

Impact: After a privileged policy rotation, registry state can diverge from the policy that actually gates transfers. Compliance operations may update an inactive policy until the stack is upgraded or reconfigured.

Proof of Concept: Add the following test to test/solace-pocs/PolicyBindingDrift.test.ts:

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";

const REGISTRY_POLICY = 2n;
const ROTATED_TOKEN_POLICY = 3n;

async function fixture() {
  const [admin, registrar, lockManager, wallet] = await ethers.getSigners();
  const tip403 = await ethers.deployContract("MockTIP403");
  const Registry = await ethers.getContractFactory("Tip20RegistryService");
  const implementation = await Registry.deploy();
  const proxy = await ethers.deployContract("ERC1967Proxy", [
    await implementation.getAddress(),
    Registry.interface.encodeFunctionData("initialize", [admin.address, await tip403.getAddress()]),
  ]);
  const registry: any = Registry.attach(await proxy.getAddress());

  await registry.setPolicyId(REGISTRY_POLICY);
  await registry.grantRole(await registry.REGISTRAR_ROLE(), registrar.address);
  await registry.grantRole(await registry.LOCK_MANAGER_ROLE(), lockManager.address);
  await registry.connect(registrar).registerInvestor("INV-1");
  await registry.connect(registrar).addWallet(wallet.address, "INV-1");

  return { registry, tip403, lockManager, wallet };
}

describe("Registry and token policy binding drift", () => {
  it("records a lock only in the registry's stale policy", async () => {
    const { registry, tip403, lockManager, wallet } = await loadFixture(fixture);

    // Model an external privileged token rotation by making policy 3 the
    // policy that token enforcement reads, while the registry remains on 2
    await tip403.modifyPolicyWhitelist(ROTATED_TOKEN_POLICY, wallet.address, true);

    await registry.connect(lockManager).lockInvestor("INV-1");

    expect(await registry.isInvestorLocked("INV-1")).to.equal(true);
    expect(await tip403.isAuthorized(REGISTRY_POLICY, wallet.address)).to.equal(false);
    expect(await tip403.isAuthorized(ROTATED_TOKEN_POLICY, wallet.address)).to.equal(true);
    await expect(registry.setPolicyId(ROTATED_TOKEN_POLICY)).to.be.revertedWithCustomError(
      registry,
      "PolicyIdAlreadySet"
    );
  });
});
```

Run with: `npx hardhat test test/solace-pocs/PolicyBindingDrift.test.ts`

The test models the out-of-scope privileged token rotation by treating policy 3 as the policy read by token enforcement while the registry remains bound to policy 2.

Recommended Mitigation: Resolve the token address and compare its effective TIP-403 binding before every policy mutation. Provide an owner-authorized upgrade or migration path that updates the registry binding only after verifying policy type and administration.

Securitize: Fixed in [PR 18](#).

Cyfrin: Verified.

7.2.2 `deploy-full.ts` uses the wrong TIP-20 `renounceRole` signature

Description: The deployment script declares and calls `renounceRole(bytes32,address)` on the native TIP-20. Tempo implements `renounceRole(bytes32)` instead ([TIP-20 specification](#)). Local tests use an OpenZeppelin-shaped mock and do not catch the mismatch.

Impact: Handover aborts after the incoming owner receives privileges but before the deployer divests, leaving two privileged accounts and an incomplete deployment.

Proof of Concept: Add the following test to `test/solace-pocs/NativeRenounceRoleAbi.test.ts`:

```
import { expect } from "chai";
import { ethers } from "ethers";

const live = process.env.TEMPO_RPC ? describe : describe.skip;

live("Native TIP-20 renounceRole ABI", () => {
  const provider = new ethers.JsonRpcProvider(process.env.TEMPO_RPC as string);
  const PATH_USD = "0x20C0000000000000000000000000000000000000000000000000000000000000";
  const PROBE = "0x0000000000000000000000000000000000000000000000000000000000000000";
  const DEFAULT_ADMIN_ROLE = ethers.ZeroHash;
  const abi = ethers.AbiCoder.defaultAbiCoder();
  const UNKNOWN_SELECTOR = "0xaa4bc69a";
  const UNAUTHORIZED = "0x82b42900";

  async function classify(signature: string, types: string[], args: unknown[]) {
    const data = ethers.id(signature).slice(0, 10) + abi.encode(types, args).slice(2);
    try {
      await provider.call({ to: PATH_USD, data, from: PROBE });
      return "PRESENT";
    } catch (error: any) {
      const revertData = String(error.data ?? error.info?.error?.data ?? "");
      if (revertData.startsWith(UNKNOWN_SELECTOR)) return "ABSENT";
      if (revertData.startsWith(UNAUTHORIZED)) return "PRESENT";
      return `UNEXPECTED:${revertData.slice(0, 18)}`;
    }
  }

  it("rejects the two-argument signature used by the deployment script", async () => {
    expect(
      await classify(
        "renounceRole(bytes32,address)",
        ["bytes32", "address"],
        [DEFAULT_ADMIN_ROLE, PROBE]
      )
    ).to.equal("ABSENT");
  });

  it("implements the one-argument native signature", async () => {
    expect(
      await classify("renounceRole(bytes32)", ["bytes32"], [DEFAULT_ADMIN_ROLE])
    ).to.equal("PRESENT");
  });

  it("retains the expected signatures for the other role operations", async () => {
```

```

const role = ethers.id("ISSUER_ROLE");
expect(await classify("grantRole(bytes32,address)", ["bytes32", "address"], [role,
  ↪ PROBE])).to.equal("PRESENT");
expect(await classify("revokeRole(bytes32,address)", ["bytes32", "address"], [role,
  ↪ PROBE])).to.equal("PRESENT");
expect(await classify("setRoleAdmin(bytes32,bytes32)", ["bytes32", "bytes32"], [role,
  ↪ DEFAULT_ADMIN_ROLE])).to.equal("PRESENT");
expect(await classify("hasRole(address,bytes32)", ["address", "bytes32"], [PROBE,
  ↪ role])).to.equal("PRESENT");
expect(await classify("hasRole(bytes32,address)", ["bytes32", "address"], [role,
  ↪ PROBE])).to.equal("ABSENT");
});
});

```

Run with: `TEMPO_RPC="https://your-tempo-mainnet-rpc" npx hardhat test test/solace-pocs/NativeRenounceRoleAbi.test.ts`

Recommended Mitigation: Use the one-argument ABI for the TIP-20 call while retaining the two-argument OpenZeppelin call for the registry and consumer. Source the native ABI from Tempo's SDK and verify every post-handoff role on a Tempo-compatible environment.

Securitize: Fixed in [PR 17](#).

Cyfrin: Verified.

7.2.3 `deploy-full.ts` predicts the TIP-403 policy ID instead of reading the receipt

Description: The deployment script obtains the next global TIP-403 policy ID with `staticCall`, sends `createPolicy`, and ignores the transaction result. Another policy creation between those calls changes the assigned ID. The deploy need not abort. FEEMGR and DEX are hardcoded and public, so whoever won the race can pre-authorize both under their own policy; the next step's guard then skips `addPlatformWallet` entirely. Step 5's registrar failures are swallowed, and the script continues through the handoff and prints DONE.

Impact: The registry is bound to a policy it cannot write to, while the race winner administers the whitelist the token enforces. Both `setPolicyId` and `changeTransferPolicyId` take the raced id, so the token points at the attacker's policy too. `setPolicyId` is one-shot and never checks `policyData(id).admin`, so the deployed implementation cannot correct the binding; recovery needs a UUPS upgrade that adds a re-point path, or a redeploy. The deploy can complete and report success in that state.

Recommended Mitigation: Read the created policy ID from the mined receipt, then verify its existence, whitelist type, and registry administrator before calling `setPolicyId`.

Securitize: Fixed in [PR 15](#).

Cyfrin: Verified.

7.2.4 `Tip20TrustService::setServiceOwner` can assign MASTER to an inert address

Description: `Tip20TrustService::setServiceOwner` writes MASTER without applying the protocol-address exclusions used by `setRole`. The current owner can transfer control to the token, registry, trust proxy, or another inert contract.

Impact: After the outgoing owner is demoted, no externally controlled account may be able to manage roles or upgrade the trust and consumer proxies.

Recommended Mitigation: Apply the same invalid-target checks to ownership transfer and use a two-step handoff in which the proposed owner must accept before the current owner is demoted.

Securitize: Fixed in [PR 16](#).

Cyfrin: Verified.

7.2.5 Tip20RegistryService::setPolicyId accepts built-in TIP-403 policies

Description: Tip20RegistryService::setPolicyId accepts policy 0 and policy 1. TIP-403 defines them as always-reject and always-allow, while custom policies start at 2 ([TIP-403 specification](#)). Before configuration, Tip20RegistryService::isAuthorized also queries policy 0 without checking _policyIdSet.

Impact: A one-shot configuration with policy 0 blocks all transfers; policy 1 makes registry whitelist changes ineffective. The registry cannot correct the binding without an upgrade.

Proof of Concept: Add the following test to test/solace-pocs/BuiltinPolicyBinding.test.ts:

```
import { expect } from "chai";
import { ethers } from "ethers";

const live = process.env.TEMPO_RPC ? describe : describe.skip;

live("TIP-403 built-in policy binding", () => {
  const provider = new ethers.JsonRpcProvider(process.env.TEMPO_RPC as string);
  const TIP403 = "0x403c000000000000000000000000000000000000000000000000000000000000";
  const abi = ethers.AbiCoder.defaultAbiCoder();
  const sampleAddresses = [
    "0x0000000000000000000000000000000000000000000000000000000000000000",
    "0x1111111111111111111111111111111111111111111111111111111111111111",
    "0x251d2711ebeb0a09fdB8992F5506f3D949175246",
  ];

  const call = async (signature: string, types: string[], args: unknown[]) =>
    provider.call({
      to: TIP403,
      data: ethers.id(signature).slice(0, 10) + abi.encode(types, args).slice(2),
    });

  const isAuthorized = async (policyId: number, account: string) =>
    BigInt(
      await call("isAuthorized(uint64,address)", ["uint64", "address"], [policyId, account])
    ) === 1n;

  it("always rejects under policy 0 and always allows under policy 1", async () => {
    for (const account of sampleAddresses) {
      expect(await isAuthorized(0, account)).to.equal(false);
      expect(await isAuthorized(1, account)).to.equal(true);
    }
  });

  it("reports the zero address as administrator of both built-in policies", async () => {
    for (const policyId of [0, 1]) {
      const [, admin] = abi.decode(
        ["uint8", "address"],
        await call("policyData(uint64)", ["uint64"], [policyId])
      );
      expect(admin).to.equal(ethers.ZeroAddress);
    }
  });

  it("reserves custom policy IDs above the built-ins", async () => {
    const nextPolicyId = BigInt(await call("policyIdCounter()", [], []));
    expect(nextPolicyId).to.be.greaterThan(1n);
  });
});
```

Run with: TEMPO_RPC="https://your-tempo-mainnet-rpc" npx hardhat test test/solace-pocs/BuiltinPolicyBinding.test.ts

Recommended Mitigation: Require a custom policy ID, verify that it exists, is a whitelist, and is administered by

the registry. Make `isAuthorized` revert or return an explicit unconfigured result until the binding is set.

Securitize: Fixed in [PR 15](#).

Cyfrin: Verified.

7.2.6 Tip20TrustService::setServiceOwner uses one-step ownership transfer

Description: `Tip20TrustService::setServiceOwner` immediately demotes the current MASTER and assigns the new address. The recipient does not accept the role, so an address typo or inaccessible account cannot be corrected afterward.

Impact: The stack can irreversibly lose abstract-role governance and upgrade authority for the trust and service-consumer proxies. The separate token, registry, and consumer `DEFAULT_ADMIN_ROLE` assignments are not transferred by this function and remain a distinct operational handoff concern.

Proof of Concept: Add the following test to `test/solace-pocs/OneStepServiceOwnerTransfer.test.ts`:

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";

const NONE = 0n;
const MASTER = 1n;
const ISSUER_ROLE = ethers.keccak256(ethers.toUtf8Bytes("ISSUER_ROLE"));
const PAUSE_ROLE = ethers.keccak256(ethers.toUtf8Bytes("PAUSE_ROLE"));
const UNPAUSE_ROLE = ethers.keccak256(ethers.toUtf8Bytes("UNPAUSE_ROLE"));
const BURN_BLOCKED_ROLE = ethers.keccak256(ethers.toUtf8Bytes("BURN_BLOCKED_ROLE"));
const REGISTRAR_ROLE = ethers.keccak256(ethers.toUtf8Bytes("REGISTRAR_ROLE"));
const LOCK_MANAGER_ROLE = ethers.keccak256(ethers.toUtf8Bytes("LOCK_MANAGER_ROLE"));
const ROLE_MANAGER_ROLE = ethers.keccak256(ethers.toUtf8Bytes("ROLE_MANAGER_ROLE"));

async function fixture() {
  const [admin, master, replacement] = await ethers.getSigners();
  const inaccessibleOwner = ethers.Wallet.createRandom().address;
  const tip403 = await ethers.deployContract("MockTIP403");
  const token: any = await ethers.deployContract("MockTIP20", [admin.address]);

  const Registry = await ethers.getContractFactory("Tip20RegistryService");
  const registryImplementation = await Registry.deploy();
  const registryProxy = await ethers.deployContract("ERC1967Proxy", [
    await registryImplementation.getAddress(),
    Registry.interface.encodeFunctionData("initialize", [admin.address, await tip403.getAddress()]),
  ]);
  const registry: any = Registry.attach(await registryProxy.getAddress());

  const Trust = await ethers.getContractFactory("Tip20TrustService");
  const trustImplementation = await Trust.deploy();
  const trustProxy = await ethers.deployContract("ERC1967Proxy", [
    await trustImplementation.getAddress(),
    Trust.interface.encodeFunctionData("initialize", [
      master.address,
      await token.getAddress(),
      await registry.getAddress(),
    ]),
  ]);
  const trust: any = Trust.attach(await trustProxy.getAddress());
  const trustAddress = await trust.getAddress();

  for (const role of [ISSUER_ROLE, PAUSE_ROLE, UNPAUSE_ROLE, BURN_BLOCKED_ROLE]) {
    await token.setRoleAdmin(role, ROLE_MANAGER_ROLE);
  }
  await token.grantRole(ROLE_MANAGER_ROLE, trustAddress);
}
```

```

    await registry.setRoleAdmin(REGISTRAR_ROLE, ROLE_MANAGER_ROLE);
    await registry.setRoleAdmin(LOCK_MANAGER_ROLE, ROLE_MANAGER_ROLE);
    await registry.grantRole(ROLE_MANAGER_ROLE, trustAddress);

    return { trust, master, replacement, inaccessibleOwner };
}

describe("One-step service-owner transfer", () => {
  it("irreversibly demotes the current MASTER before recipient acceptance", async () => {
    const { trust, master, replacement, inaccessibleOwner } = await loadFixture(fixture);

    expect(await trust.getRole(master.address)).to.equal(MASTER);
    await trust.connect(master).setServiceOwner(inaccessibleOwner);

    expect(await trust.getRole(master.address)).to.equal(NONE);
    expect(await trust.getRole(inaccessibleOwner)).to.equal(MASTER);
    await expect(
      trust.connect(master).setServiceOwner(replacement.address)
    ).to.be.revertedWithCustomError(trust, "InsufficientRole");
  });
});

```

Run with: `npx hardhat test test/solace-pocs/OneStepServiceOwnerTransfer.test.ts`

Recommended Mitigation: Use a two-step pendingOwner and acceptance flow. Keep the current MASTER active until the proposed owner accepts, and apply the protocol-address target checks before nomination.

Securitize: Fixed in [PR 16](#).

Cyfrin: Verified.

7.2.7 Tip20RegistryService::registerInvestorWithWallets exceeds Tempo's transaction gas cap

Description: Tip20RegistryService::registerInvestorWithWallets permits a registrar to add up to 100 wallets atomically. For a new investor with a short ID, each wallet creates a _walletInvestor slot, an _investor-Wallets element, and a TIP-403 membership slot; _investorExists and the array length create two more slots. Registering 100 wallets therefore creates at least 302 slots. Tempo charges 250,000 gas for each new storage slot and caps a transaction at 30 million gas, so storage creation alone requires at least 75.5 million gas before call, loop, event, and calldata costs ([Tempo EVM compatibility documentation](#)).

Impact: The documented one-transaction holder-creation flow cannot onboard the maximum supported wallet set on Tempo. A registrar or provider that submits such a request receives an out-of-gas failure even though the input satisfies MAX_WALLETS_PER_INVESTOR.

Recommended Mitigation: Keep MAX_WALLETS_PER_INVESTOR as the total per-investor bound, but add a smaller per-transaction wallet batch limit measured under Tempo's gas schedule. Register the investor first and append larger wallet sets across multiple transactions, or make the provider chunk wallet additions while preserving explicit failure handling.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.2.8 loadPk reads deployer keys from plaintext sources

Description: Deployment and operational scripts deploy-token.ts, register-investor.ts, mint.ts, deploy-full.ts, register-and-mint.ts use function loadPk to load the deployer private key from DEPLOYER_PK and, when it is unset, from an unencrypted .deployer.key file. Ignoring the file in version control does not protect it from access on a developer workstation, CI runner, backup, or log collection system.

Impact: An attacker who obtains the plaintext key through a secondary environment compromise can sign transactions as the deployer and exercise any authority held by that address. This is not a direct on-chain exploit, but

compromises deployment and operational trust boundaries.

Recommended Mitigation: Remove support for plaintext key files. Use an encrypted keystore or a managed, hardware-backed, or remote signer so the private key is not exposed to the deployment process. Configure CI to use a least-privileged signing integration, rotate any key previously stored in plaintext, and remove residual copies from developer and CI environments.

Securitize: Cyfrin:

7.3 Informational

7.3.1 Tip20TrustService::_project can revoke unmanaged native grants

Description: Tip20TrustService::_project revokes every native role in an outgoing abstract projection without knowing who granted it. A lateral role holder can assign and then remove an abstract role from an account that already held the same native permission directly, deleting the unmanaged grant.

Impact: A trust transition can unexpectedly remove break-glass or manually provisioned permissions. A MASTER can recover them, so this does not permanently seize governance.

Proof of Concept: Add the following test to test/solace-pocs/UnmanagedNativeGrantRevocation.test.ts:

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";

const NONE = 0;
const MASTER = 1;
const ISSUER = 2;
const REGISTRAR_ROLE = ethers.keccak256(ethers.toUtf8Bytes("REGISTRAR_ROLE"));
const LOCK_MANAGER_ROLE = ethers.keccak256(ethers.toUtf8Bytes("LOCK_MANAGER_ROLE"));
const ISSUER_ROLE = ethers.keccak256(ethers.toUtf8Bytes("ISSUER_ROLE"));
const PAUSE_ROLE = ethers.keccak256(ethers.toUtf8Bytes("PAUSE_ROLE"));
const UNPAUSE_ROLE = ethers.keccak256(ethers.toUtf8Bytes("UNPAUSE_ROLE"));
const BURN_BLOCKED_ROLE = ethers.keccak256(ethers.toUtf8Bytes("BURN_BLOCKED_ROLE"));
const ROLE_MANAGER_ROLE = ethers.keccak256(ethers.toUtf8Bytes("ROLE_MANAGER_ROLE"));

async function fixture() {
  const [admin, master, issuer, walletRegistrar] = await ethers.getSigners();
  const tip403 = await ethers.deployContract("MockTIP403");
  const token: any = await ethers.deployContract("MockTIP20", [admin.address]);

  const Registry = await ethers.getContractFactory("Tip20RegistryService");
  const registryImplementation = await Registry.deploy();
  const registryProxy = await ethers.deployContract("ERC1967Proxy", [
    await registryImplementation.getAddress(),
    Registry.interface.encodeFunctionData("initialize", [admin.address, await tip403.getAddress()]),
  ]);
  const registry: any = Registry.attach(await registryProxy.getAddress());
  await registry.grantRole(REGISTRAR_ROLE, walletRegistrar.address);

  const Trust = await ethers.getContractFactory("Tip20TrustService");
  const trustImplementation = await Trust.deploy();
  const trustProxy = await ethers.deployContract("ERC1967Proxy", [
    await trustImplementation.getAddress(),
    Trust.interface.encodeFunctionData("initialize", [
      master.address,
      await token.getAddress(),
      await registry.getAddress(),
    ]),
  ]);
  const trust: any = Trust.attach(await trustProxy.getAddress());
  const trustAddress = await trust.getAddress();

  for (const role of [ISSUER_ROLE, PAUSE_ROLE, UNPAUSE_ROLE, BURN_BLOCKED_ROLE]) {
    await token.setRoleAdmin(role, ROLE_MANAGER_ROLE);
  }
  await token.grantRole(ROLE_MANAGER_ROLE, trustAddress);
  await registry.setRoleAdmin(REGISTRAR_ROLE, ROLE_MANAGER_ROLE);
  await registry.setRoleAdmin(LOCK_MANAGER_ROLE, ROLE_MANAGER_ROLE);
  await registry.grantRole(ROLE_MANAGER_ROLE, trustAddress);
  await trust.connect(master).setRole(issuer.address, ISSUER);
}
```



```

    return { trust, registry, admin, master, issuer, walletRegistrar };
}

describe("Unmanaged native grant revocation", () => {
  it("lets an issuer erase a registrar grant that the trust did not create", async () => {
    const { trust, registry, issuer, walletRegistrar } = await loadFixture(fixture);

    expect(await registry.hasRole(REGISTRAR_ROLE, walletRegistrar.address)).to.equal(true);
    expect(await trust.getRole(walletRegistrar.address)).to.equal(NONE);
    expect(await trust.getRole(issuer.address)).to.equal(ISSUER);
    expect(await trust.getRole(issuer.address)).to.not.equal(MASTER);

    await trust.connect(issuer).setRole(walletRegistrar.address, ISSUER);
    await trust.connect(issuer).removeRole(walletRegistrar.address);

    expect(await registry.hasRole(REGISTRAR_ROLE, walletRegistrar.address)).to.equal(false);
  });

  it("prevents the token owner from restoring the grant directly", async () => {
    const { trust, registry, admin, master, walletRegistrar } = await loadFixture(fixture);

    await trust.connect(master).setRole(walletRegistrar.address, ISSUER);
    await trust.connect(master).removeRole(walletRegistrar.address);

    await expect(registry.connect(admin).grantRole(REGISTRAR_ROLE,
      ↪ walletRegistrar.address)).to.be.reverted;
  });
});

```

Run with: `npx hardhat test test/solace-pocs/UnmanagedNativeGrantRevocation.test.ts`

Recommended Mitigation: Treat direct native grants as an explicit break-glass state and reconcile them before trust transitions. If coexistence is required, track which grants the trust introduced and revoke only those grants, with tests for pre-existing native permissions.

Securitize: Acknowledged.

7.3.2 Tip20RegistryService::isAuthorized documentation does not match policy state

Description: Tip20RegistryService::isAuthorized returns the live TIP-403 authorization bit. A platform wallet can therefore return true without an investor, while a staged wallet can return false after its investor is unlocked. The interface instead describes the result as equivalent to being registered and not locked.

Recommended Mitigation: Document isAuthorized as a live policy-membership query. If consumers also need identity status, expose or use a separate predicate for registration and lock state.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.3.3 Tip20RegistryService events do not expose effective wallet authorization

Description: Registry wallet and lock events describe identity changes, not the final TIP-403 authorization set. In particular, InvestorFullyUnlocked can be emitted while wallets staged during the lock remain unauthorized. An indexer cannot reconstruct policy membership from registry events alone.

Recommended Mitigation: Document TIP-403 events and direct policy queries as the authorization source of truth. If registry-only reconstruction is required, emit a per-wallet authorization event whenever a policy bit changes and clarify the meaning of InvestorFullyUnlocked.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.3.4 `Tip20RegistryService::setServiceConsumer` missing event

Description: `Tip20RegistryService::setServiceConsumer` changes an important storage variable but does not emit an event.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.3.5 `Tip20TrustService::_setRole` emits abstract-role events after native calls

Description: `Tip20TrustService::_setRole` updates storage, performs native token and registry calls, and only then emits its abstract-role events. This differs from the documented event-before-external-call convention. Transaction reversion remains atomic, so contract state is not corrupted.

Recommended Mitigation: Either document the existing log order or emit the abstract-role change before native external calls, preserving checks-effects-interactions and relying on transaction reversion to remove logs on failure.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.3.6 `Tip20RegistryService::registerInvestorWithWallets` can retain stale proof metadata

Description: Idempotent `Tip20RegistryService::registerInvestorWithWallets` can update an attribute's value and expiry while preserving its existing proof hash. This is correct if the hash identifies independent evidence, but ambiguous if it commits to the prior attribute tuple.

Recommended Mitigation: Define what the proof hash attests. If it commits to value or expiry, require a replacement hash whenever those fields change. If it identifies independent evidence, document the preservation behavior explicitly.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.3.7 `deploy-full.ts` permits overlapping operational role addresses

Description: The script takes three addresses from the environment — issuer, transfer agent and token owner — and checks only that the issuer and transfer agent aren't the deployer (:328-333). Nothing compares them to each other, and nothing checks the token owner at all.

Four overlaps are possible, and they don't behave the same way.

Issuer and transfer agent set to the same address. Abstract roles are exclusive, so `setRole(x, ISSUER)` followed by `setRole(x, TRANSFER_AGENT)` (:216-221) replaces the first rather than adding to it. That account is left without the issuer role, so it can't mint, onboard investors or lock them — only the token owner can, through MASTER. So you lose the separate issuer you configured, not the capability. The script still prints success. It's recoverable: the token owner grants ISSUER back in one transaction.

Issuer or transfer agent set to the token owner's address. Harmless. `setServiceOwner` runs last (:241) and promotes that account to MASTER, whose native grants cover everything ISSUER and TRANSFER_AGENT project. The operator ends up with more authority, not less.

Token owner set to the deployer's address. Here the effect is permanent. The handover grants DEFAULT_ADMIN_ROLE to the token owner on the token, registry and ServiceConsumer (:244-246), then has the deployer renounce its own on the same three (:248-250). Same address, so the grant does nothing — the account already holds the role — and the renounce then removes it. All three end up with nobody holding admin.

That role administers itself, so once there are no holders it can never be granted again, and the registry's upgrade function is gated on it (`Tip20RegistryService.sol:132`). The deployer keeps MASTER on the trust, so lock and unlock still work, but the script renounces REGISTRAR_ROLE one line after the handoff re-granted it, so registering

new investors reverts from the moment the deploy ends. That part is repairable by handing MASTER through a second key and back. The lost DEFAULT_ADMIN_ROLE is not. Confirmed by running it.

The script already carries a comment at :313-319 explaining why TOKEN_OWNER must not default to the deployer, so this class of mistake was considered — the checks just don't cover it.

Recommended Mitigation: Extend the check at :328-333 so it rejects two configurations before any transaction is sent: the token owner equal to the deployer, and the issuer equal to the transfer agent unless that address is also the token owner. Anything the token owner shares an address with is safe, because it ends up as MASTER either way.

Securitize: Fixed in [PR 17](#).

Cyfrin: Verified.

7.3.8 `deploy-full.ts` blindly retries one-shot deployment operations

Description: The deployment script uses `retry()` to resubmit a transaction after any error, including errors returned while waiting for the receipt. Some one-shot operations have read guards, but those guards are evaluated before entering `retry()` rather than on every attempt.

If the original transaction is mined successfully but receipt retrieval fails, the next attempt repeats the already-completed operation. One-shot calls such as `setRegistry`, `setTrust`, `setServiceConsumer`, `setPolicyId`, and `setRole` then revert instead of recognizing that their postcondition has already been satisfied.

Retried `createPolicy` may also create an unused additional policy.

Impact: A transient RPC or receipt-retrieval failure can cause deployment to abort after some operations have completed successfully. This may leave a partially handed-off stack requiring manual intervention or redeployment and may create unused contracts or policies.

Recommended Mitigation: Evaluate each operation's postcondition inside the `retry` callback. After broadcasting a transaction, preserve its hash and attempt to recover its receipt before submitting another transaction.

Securitize: Acknowledged.

7.3.9 `deploy-full.ts` suppresses investor onboarding failures

Description: The deployment script appends an empty catch handler to investor registration and wallet addition. Any revert, including policy or authorization failure, is ignored and the script continues to print success.

Impact: A deployment can finish with the expected investor or wallet missing from the registry and TIP-403 policy.

Recommended Mitigation: Remove the blanket catch. Use read guards for expected already-completed states, propagate every other error, and verify the investor mapping and policy authorization before reporting success.

Securitize: Fixed in [PR 17](#).

Cyfrin: Verified.

7.3.10 `deploy-full.ts` does not verify the target Tempo network

Description: `deploy-full.ts` sends irreversible deployment and configuration transactions without checking `eth_chainId` or validating the identity of the configured TIP20Factory, TIP-403 registry, and pathUSD addresses. The script defaults to the Moderato RPC and testnet explorer. Even when an operator supplies a mainnet RPC, its final MetaMask instruction still reports testnet chain ID 42431 rather than Tempo mainnet chain ID 4217 ([Tempo network documentation](#)).

Recommended Mitigation: Require an explicit deployment profile and expected chain ID. Before the first transaction, assert the connected chain and verify the required native predeploy identities and selector behavior. Derive the displayed chain ID and explorer from the validated profile instead of hard-coding testnet values.

Securitize: Fixed in [PR 17](#).

Cyfrin: Verified.

7.3.11 `ITip20TrustService` **misdocuments MASTER native permissions**

Description: `ITip20TrustService` says MASTER receives no native grants. `Tip20TrustService::_project`, the README, and the protocol specification instead define MASTER as the union of all token and registry permissions. The implementation is correct; the published interface understates the strongest key's authority.

Recommended Mitigation: Update the interface role table and `setServiceOwner` documentation to list the complete MASTER projection. Leave NONE as the only role with no native grants.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.3.12 `Tip20RegistryService::addPlatformWallet` **has an unreachable duplicate check**

Description: `Tip20RegistryService::addPlatformWallet` calls `_assertRegisterableWallet`, which already rejects an existing platform wallet. The following duplicate check tests the same condition and can never execute.

Recommended Mitigation: Remove the second check.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.3.13 `Tip20RegistryService` **storage-gap comment has the wrong slot count**

Description: The storage comment says the contract declares 9 slots, but its own enumeration covers slots 0 through 9. The 10 declared slots plus the 40-slot gap still preserve the intended 50-slot window, so the implementation is correct.

Recommended Mitigation: Change the comment from 9 declared slots to 10.

Securitize: Fixed in [PR 14](#).

Cyfrin: Verified.

7.3.14 `Tip20RegistryService::registerInvestorWithWallets, isAuthorized` **mishandle Tempo virtual addresses**

Description: Tempo TIP-20 resolves a virtual recipient address to its registered master before recipient validation and transfer-policy authorization, while TIP-403 rejects virtual aliases as policy members and requires configuration of the resolved master. `Tip20RegistryService::registerInvestorWithWallets` forwards the supplied wallet unchanged to `ITIP403::modifyPolicyWhitelist`, so onboarding a virtual alias reverts. Conversely, after the master is registered and whitelisted, a TIP-20 transfer to its virtual alias can succeed after resolution while `Tip20RegistryService::isAuthorized` forwards the raw alias to `ITIP403::isAuthorized` and returns false ([TIP-20 specification](#), [TIP-403 specification](#)).

Impact: Integrations that accept Tempo virtual addresses either fail holder onboarding or report an authorization result that differs from the native token's effective recipient authorization.

Recommended Mitigation: Document on `registerInvestorWithWallets` and `isAuthorized` that callers must supply resolved master addresses rather than virtual aliases.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.

7.3.15 deploy-full.ts can never execute mint step

Description: deploy-full.ts signs every transaction with the deployer, so its mint branch is conditioned on the configured issuer being the deployer:

```
if (issuerWallet.toLowerCase() === deployer.address.toLowerCase()) {  
  await retry(async () => {  
    await (await tok.mint(INVESTOR_WALLET, ethers.parseUnits(MINT_AMOUNT, dec))).wait();  
  }, 'mint');  
}
```

However, the configuration validation immediately before deployTrustStack rejects that exact relationship:

```
if (  
  issuerWallet.toLowerCase() === deployer.address.toLowerCase() ||  
  transferAgentWallet.toLowerCase() === deployer.address.toLowerCase()  
) {  
  throw new Error('ISSUER_WALLET / TRANSFER_AGENT_WALLET must not be the deployer (it holds MASTER).');  
}
```

Consequently, every configuration that reaches the mint condition necessarily makes it false. A configuration that would make it true is rejected earlier.

Impact: Dead code and a mismatch between the script's full-deployment behavior and its executable behavior.

Recommended Mitigation: Consider removing the unreachable mint branch and MINT_AMOUNT configuration from deploy-full.ts and document the required post deployment issuer transaction.

Securitize: Fixed in [PR 17](#).

Cyfrin: Verified.

7.3.16 Tip20RegistryService::initialize does not reject a zero admin

Description: Tip20ServiceConsumer::initialize (:72) and Tip20TrustService::initialize (:90) both revert ZeroAddress on a zero admin. Tip20RegistryService::initialize (:127) grants DEFAULT_ADMIN_ROLE unchecked, leaving a proxy nobody can configure or upgrade.

Recommended Mitigation: Add the same check to Tip20RegistryService::initialize.

Securitize: Fixed in [PR 19](#).

Cyfrin: Verified.